Rețele de calculatoare

rețea = grup sau sistem de lucruri / persoane interconectate
rețea de calculatoare = un grup de calculatoare interconectate

Dispozitive de rețea

↳ folosite pt a conecta și gestiona traficul de date între calculatoare sau alte dispozitive de rețea.

Hub (Concentrator) : one collision → / ←

- de obicei conectează dispozitive în topologie de stea
- când un dispozitiv trimite date, hub-ul trimite către toate dispozitivele celelalte
- dispozitiv de nivel fizic (nivel 1 din modelul OSI)
  ↳ nu distinge dintre destinatarul corect și celelalte dispozitive
- wasted band with, security risks, ineffficient

Bridge (Pod de rețea) ⟺

- conectează două sau mai multe segmente de rețea și controlează traficul dintre ele
- funcționează la nivelul de legătură de date (nivel 2 din OSI) și poate decide care pachete de date trebuie să treacă între segmentele de rețea pe baza adreselor MAC

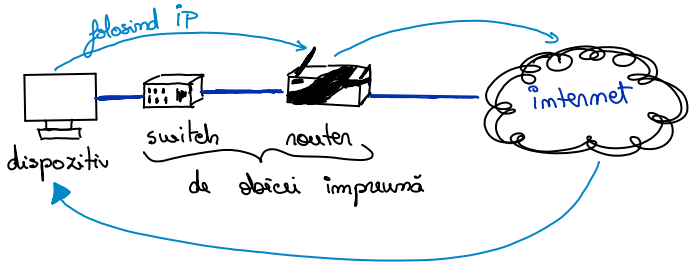
→ filtrează traficul inutil pentru a reduce coliziunile și congestiile de date
→ ! dezavantaj: nu poate face rutare între rețele diferite (subrețele), deci este mai limitat decât un router

Switch : full-duplex

- similar cu un hub $\xrightarrow{\text{dar}}$ mai eficient
- funcționează la nivelul de legătură de date (nivel 2 din OSI) și folosește adrese MAC pt a direcționa traficul
  ↳ trimite pachetele de date direct la dispozitivul destinat
- saves band width, increased security, efficient

Router

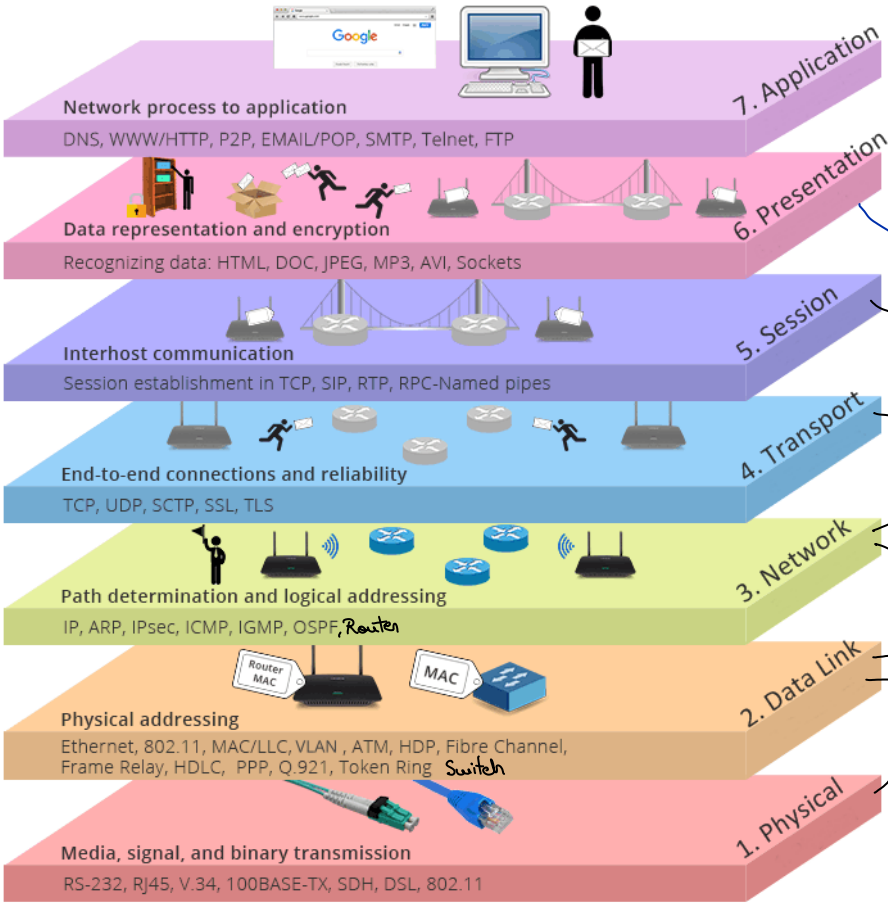
- conectează și gestionează traficul între diferite rețele, lucrând la nivelul rețea (nivel 3 din OSI)
- folosește adrese IP pt a decide cea mai bună cale de transmitere a datelor între dispozitive
- poate conecta mai multe tipuri de rețea + poate aplica reguli avansate de rutare (firewall, control acces)
- mai puține porturi



OSI Model

↳ Open System Interconnection este un cadru conceptual utilizat pt a descrie și înțelege funcționalitatea rețelelor ↳ NOT USED NOW

→ pt a standardiza



→ interacțiuni cu aplicația

↳ se ocupă cu formatarea datelor pt a le face compatibile cu dispozitivele și aplicațiile din rețea

↳ compresie, criptare, operații de prelucrare a datelor

↳ stabilește și închide sesiunile de comunicare între dispozitive

↳ asigură comunicarea fiabilă și controlul fluxului între dispozitive (protocoale < TCP / UDP)

↳ routarea datelor între rețele diferite

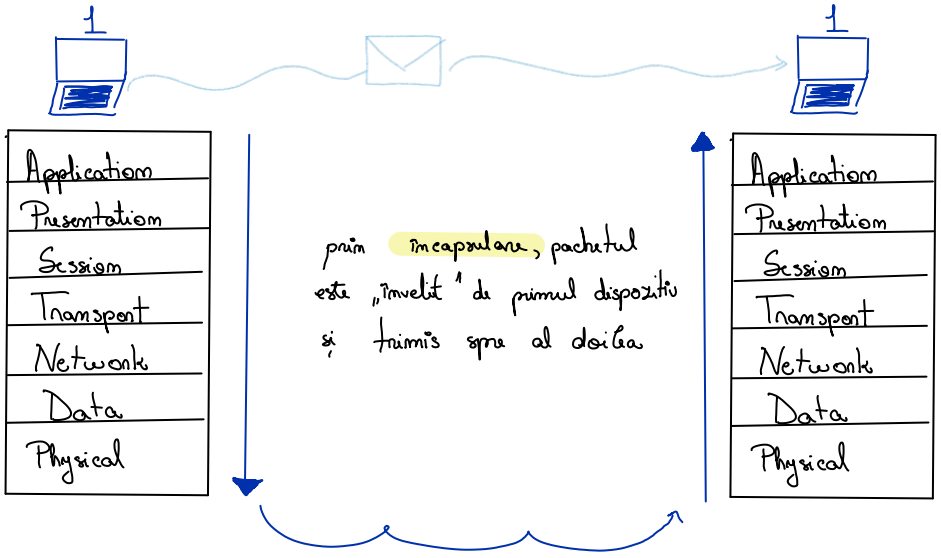
↳ utilizează adrese IP pt a dirija pachetele către destinația corectă

↳ transmite date între disp conectate la același LAN

↳ se adună MAC addresses și se efectuează verif de erori și asigură trans. fiabilă

↳ componente fizice: cabluri, conectori, network interface, hubs;

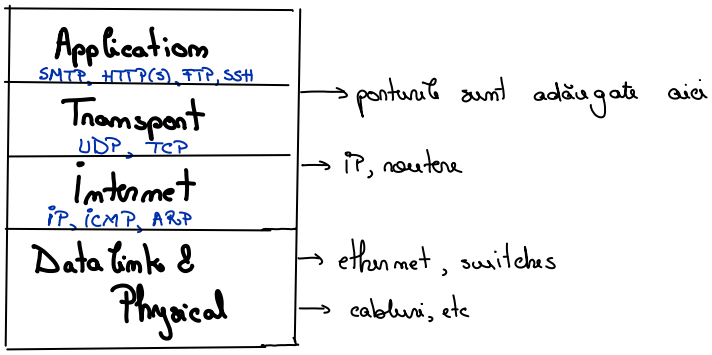
Procesul de trimitere



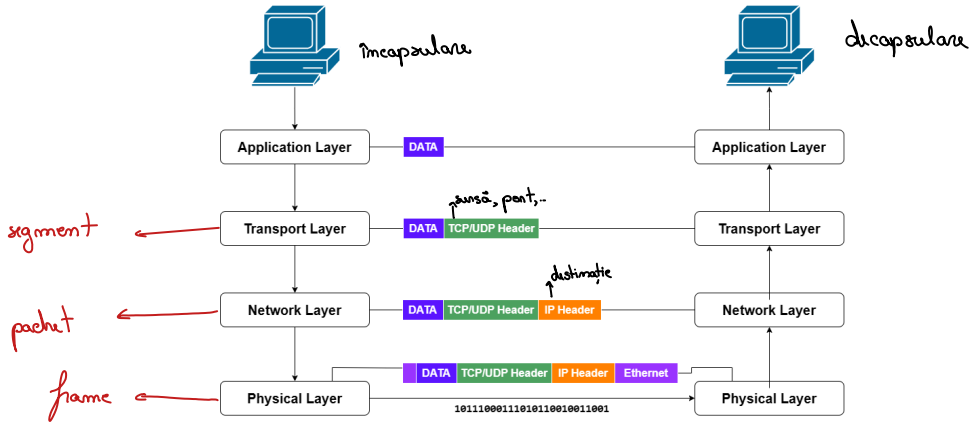
Troubleshooting: dacă există erori se verifică fiecare nivel ca să ne asigurăm că funcționează corect

TCP/IP Model

↳ folosit acum



Modul trimiterii



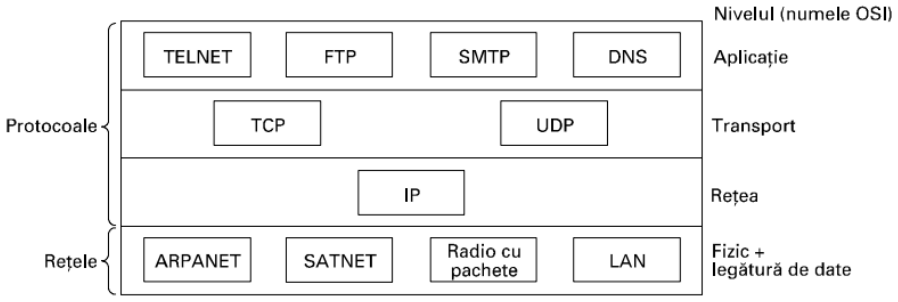


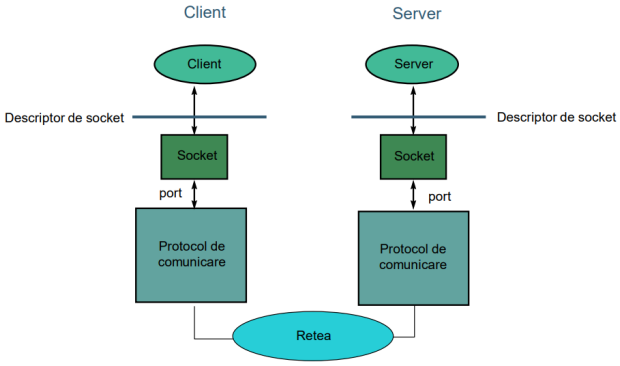
Fig. 1-22. Protocoale și rețele din modelul TCP/IP inițial.

Sockets

= comunicare între procese de pe calculatoare diferite (inclusiv SO diferite)

- se utilizează pentru transmiterea și recepționarea de pachete în rețea
- facilitează dezvoltarea de aplicații în rețea
- 2 tipuri de protocoale: UDP, TCP

Comunicare prin sockets



Tipuri de sockets

Socket stream

- serviciu orientat conexiune
- date recepționate în ordinea transmiterii
- protocol TCP

* analogie: aparat telefonic

Socket datagram

- serviciu fără conexiune
- nu garantează recepționarea datelor
- datele pot ajunge în altă ordine decât în cea în care au fost transmise
- protocol UDP

* analogie: cutie poștală

Rețele de calculatoare

Curs 1

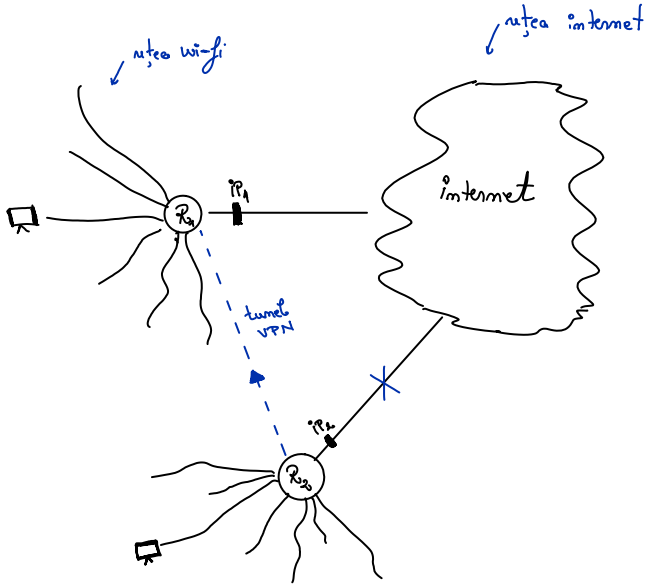
Notă: 40% laborator + 60% examen scriere + bonusuri laborator

Site: ubuntu

Principii ale rețelelor de calculatoare

- mecanismul de dirijare
- mecanismul de adresare
- mecanismul DNS (Domain Name Service)
- stiva de protocoale TCP / IP → RFC-uri
- meca nism/ paradigmă client - server

OPEN WRT

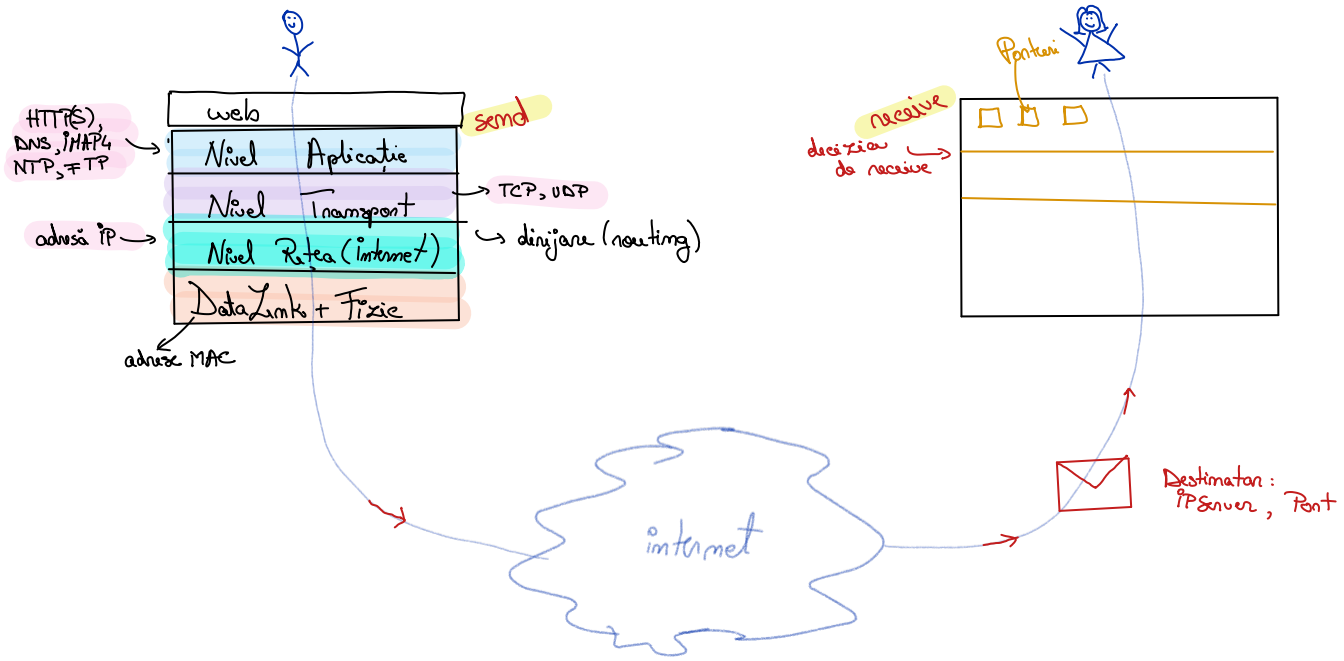


cmd → ipconfig

Wi-fi calling

Obs
browser = client

Stivă TCP / IP



OSI & TCP/IP Protocol-Stacks and Protocols

OSI	TCP/IP	Protocols
Application	Application	SMTP,FTP, HTTP,POP3, IMAP4,SNMP
Presentation		
Session		
Transport	Transport	TCP & UDP
Networking	Networking	IP
Datalink	Datalink And Physical	Ethernet
Physical		

NTP : se bazează pe UDP, pt reglarea unui ceas

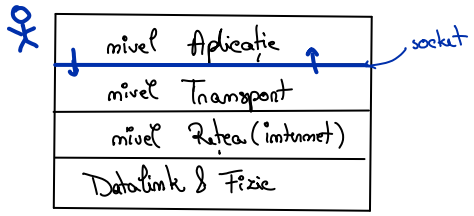
Curs 2

Socket-uri = mecanism de comunicare BIDIRECȚIONAL între procese

(exemple): CTRL-C, CTRL-V, pipe, fifo, cozi de mesaje, memorie partajată

→ avantaj sockets: nu trebuie să ruleze pe același calculator/sistem, pot fi SO-uri diferite dar trebuie să fie conectate la internet

Stiva TCP/IP



send(...) ⟶ recv(..)

– 2 tipuri: socket-uri UDP, socket-uri TCP

verificările transmisiei datelor sunt făcute la nivel transport

Socket-uri TCP

Client —localizează prin IP și port-ul S→ Server (trebuie pornit primul)

IP: locație, aparține unui device care se conectează la internet (~ casa)
Port: localizăm procesul pe care-l vrem (~ camera)

Servere: – iterative: tratează clienții în ordine câte unul pe rând
– concurente: tratează mai mulți clienți paralel (cu threads, procese etc)

– clientul trebuie să știe IP-ul serverului

*
www.emag.ro ⟶ DNS va da IP-ul
⟶ port standard prin http, https, / portul se deduce din protocol

pt Linux: /etc/services → porturile cele mai importante / frecvente

Linux: netstat -lnp
Windows: netstat -aom

Server TCP

```c
int main() {
    int sock;
    struct sockaddr_in server;
    char message[100], server_reply[200];

    // creeaza socket
    sock = socket(AF_INET, SOCK_STREAM, 0);
    if (sock == -1) {
        printf("eroare la crearea socketului\n");
        return 1;
    }

    server.sin_addr.s_addr = inet_addr("127.0.0.1");  // setez IP
    server.sin_family = AF_INET;  // setez tipul de adresa
    server.sin_port = htons(1234);  // setez portul

    // conectare la server
    if (connect(sock, (struct sockaddr *)&server, sizeof(server)) < 0) {
        printf("eroare la conectare\n");
        return 1;
    }

    printf("introduceti mesajul: ");
    scanf("%s", message);  // citeste mesajul de la utilizator

    // trimite mesajul
    send(sock, message, strlen(message), 0);

    // primeste raspunsul
    recv(sock, server_reply, sizeof(server_reply), 0);
    printf("raspunsul de la server: %s\n", server_reply);

    close(sock);  // inchide socketul
    return 0;
}
```

IP + port

Client TCP

```python
import socket

# configurarea serverului
server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)  # creeaza un socket TCP
server_socket.bind(('127.0.0.1', 1234))  # leaga socket-ul la IP si port
server_socket.listen(1)  # asculta pentru o conexiune
print("serverul asteapta conexiuni...")

while True:
    client_socket, addr = server_socket.accept()  # accepta conexiunea client
    print(f"client conectat de la {addr}")

    input_data = client_socket.recv(1024).decode()  # primeste date de la client
    print(f"date primite: {input_data}")

    response = f"echo: {input_data}"  # pregateste raspunsul
    client_socket.send(response.encode())  # trimite raspunsul inapoi clientului

    client_socket.close()  # inchide socket-ul client
```

IP Port

→ specific pt server

Special and reserved addresses
There are several special addresses:

INADDR_LOOPBACK (127.0.0.1)
always refers to the local host via the loopback device;

INADDR_ANY (0.0.0.0)
means any address for socket binding;

INADDR_BROADCAST (255.255.255.255)
has the same effect on bind(2) as **INADDR_ANY** for historical reasons. A packet addressed to **INADDR_BROADCAST** through a socket which has **SO_BROADCAST** set will be broadcast to all hosts on the local network segment, as long as the link is broadcast-capable.

apelul sistem

c = socket(AF_INET, SOCK_STREAM, 0)

descriptor adresa structurii

connect(c , { AF_NET, IPServer, portServer } , sizeof(structura))

= -1 ⇒ fail

? < 0 ⇒ fail

local host: 127.0.0.1

bind(s, { - INADDR_ANY ← adresă specială (0.0.0.0) "fă-te server pe portul 5000 și pe orice IP al device-ul"
- port (ex. 5000) → poate fi specificat punctual 'IP-ul')

→ dacă e bind pe același port dar IPs diferite atunci se poate ocupa portul

→ nu e nevoie de bind la client pt că el ocupă un port random din cele rămase

Obs

Într-o principiu, două procese nu pot face bind pe același port, dar există excepții

Un device poate avea mai multe adrese IP

Într-o situație pot 2 procese să ocupe același port? 1) dacă fac bind pe adrese IP diferite, dar același port
2) când e Server UDP și receiver TCP

client:

c = socket(AF_INET, SOCK_STREAM, 0)

- c < 0 eroare
- AF_INET → constantă simbolică socket IPv4

server:

s = socket (AF_INET, SOCK_STREAM, 0)

connect (s, { - IP, - port }, sizeof(struct))

- { - IP, - port } → o structură → se dau ca param adr. struct
- < 0 ⇒ eroare

→ Linux: ps aux f

Curs 3

client.c

```c
#include <sys/types.h>
#include <sys/socket.h>
#include <stdio.h>
#include <netinet/in.h>
#include <netinet/ip.h>
#include <string.h>

int main() {
  int c;
  struct sockaddr_in server;
  uint16_t a, b, suma;

  c = socket(AF_INET, SOCK_STREAM, 0);
  if (c < 0) {
    printf("Eroare la crearea socketului client\n");
    return 1;
  }

  memset(&server, 0, sizeof(server));
  server.sin_port = htons(1234);
  server.sin_family = AF_INET;
  server.sin_addr.s_addr = inet_addr("127.0.0.1");

  if (connect(c, (struct sockaddr *) &server, sizeof(server)) < 0) {
    printf("Eroare la conectarea la server\n");
    return 1;
  }

  printf("a = ");
  scanf("%hu", &a);
  printf("b = ");
  scanf("%hu", &b);
  a = htons(a);
  b = htons(b);
  send(c, &a, sizeof(a), 0);
  send(c, &b, sizeof(b), 0);
  recv(c, &suma, sizeof(suma), 0);
  suma = ntohs(suma);
  printf("Suma este %hu\n", suma);
  close(c);
}
```

serverConcurent.c

```c
#include <sys/types.h>
#include <sys/socket.h>
#include <stdio.h>
#include <netinet/in.h>
#include <netinet/ip.h>
#include <string.h>

void deservire_client(int c) {
  // deservirea clientului
  uint16_t a, b, suma;
  recv(c, &a, sizeof(a), MSG_WAITALL);
  recv(c, &b, sizeof(b), MSG_WAITALL);
  a = ntohs(a);
  b = ntohs(b);
  suma = a + b;
  suma = htons(suma);
  send(c, &suma, sizeof(suma), 0);
  close(c);
  // sfarsitul deservirii clientului;
}

int main() {
  int s;
  struct sockaddr_in server, client;
  int c, l;

  s = socket(AF_INET, SOCK_STREAM, 0);
  if (s < 0) {
    printf("Eroare la crearea socketului server\n");
    return 1;
  }

  memset(&server, 0, sizeof(server));
  server.sin_port = htons(1234);
  server.sin_family = AF_INET;
  server.sin_addr.s_addr = INADDR_ANY;

  if (bind(s, (struct sockaddr *) &server, sizeof(server)) < 0) {
    printf("Eroare la bind\n");
    return 1;
  }

  listen(s, 5);

  l = sizeof(client);
  memset(&client, 0, sizeof(client));

  while (1) {
    c = accept(s, (struct sockaddr *) &client, &l);
    printf("S-a conectat un client.\n");
    if (fork() == 0) { // fiu
      deservire_client(c);
      return 0;
    }
    // se executa doar in parinte pentru ca fiul se termina mai sus din cauza exit-ului
  }
}
```

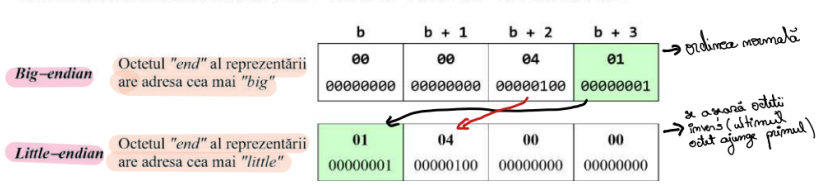
Client

Server

$(173)_{10} = (AD)_{16}$

AD	00

```
uint16_t  n = 173
n = htons(n); // traduce din reprezentarea mea
              // în reprezentarea rețea
send(c, &n, sizeof(n));  ------------>  recv(c', &n, sizeof(n), MSG_WAITALL)
                                        n = ntohs(n)
```



`htons()`, `htonl()`, `ntohs()`, `ntohl()` – utilizate pentru a converti valorile între ordinea de octeți a gazdei (host byte order) și ordinea de octeți a rețelei (network byte order)

`htonl(uint32_t hostlong)` → din host byte order în network byte order

`htons(uint16_t hostshort)` → din host byte order în network byte order

`ntohl(uint32_t netlong)` → din network byte order în host byte order

`ntohs(uint16_t netshort)` → din network byte order în host byte order

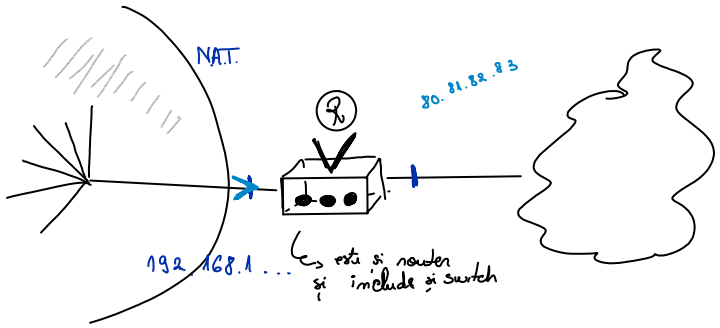
Network byte order = Big Endian (most significant byte first)

↳ dacă host folosește Big-Endian nu se face nicio modificare

Curs 4

netstat -lnp
↳ toate procesele server

mod NAT, mod Bridge



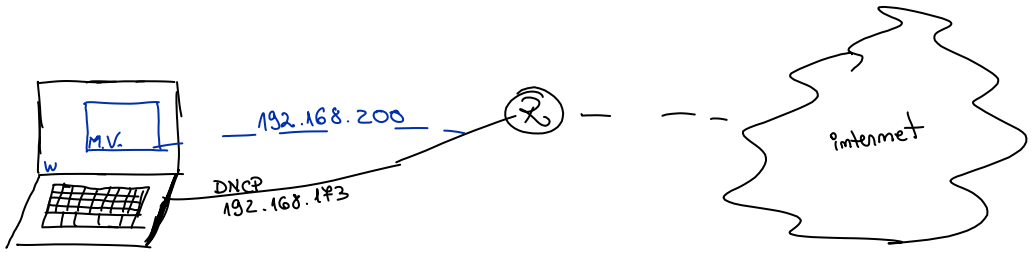
R: IP → în rețea
↓
internă: 192.168.1...

default gate = ...
adresă IP falsă = meu -> router bătă

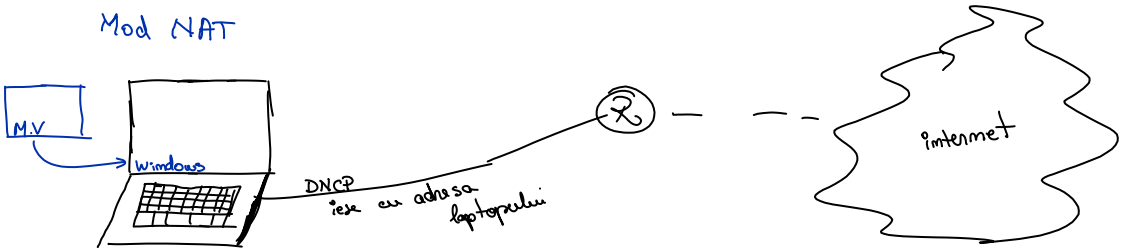
IP_S = 192.168.1.73
IP_D = 1.2.3.4

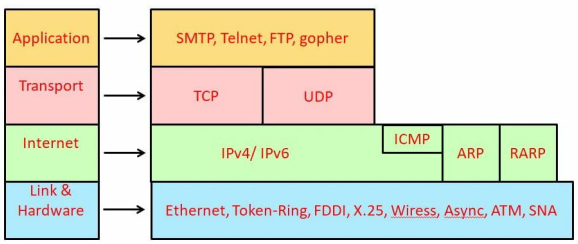
Network address translation (NAT)

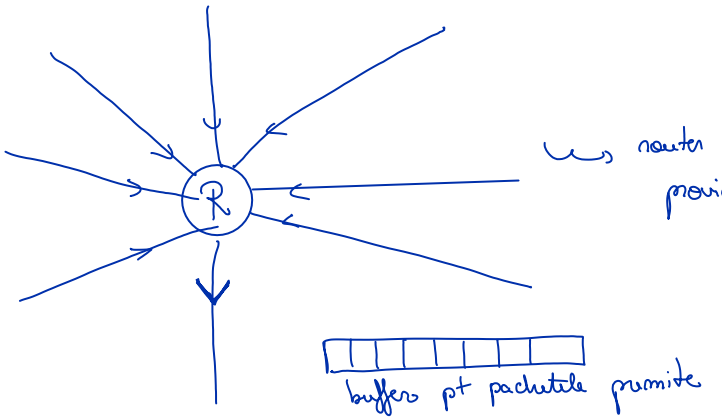
— pt a comunica între 2 calculatoare fără internet ne trebuie doar switch



Mod NAT







↳ router local pt mai multi provideri

Socket-uri UDP

↳ posibil să piardă date, unreliable, e mai rapid pt că nu face verificări

client = emițător

socket()
~~connect()~~
sendto()
↳ se modifică param

server = receptor

socket()
bind()
~~listen()~~
~~accept()~~
recvfrom()

UDP: când nu ne interesează integritatea datelor ci rapiditatea transmisiunii

Socket-uri UDP

Server UDP - secvențial

```
import socket

# configurarea serverului
server_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)  # creeaza un socket
UDP
server_socket.bind(('127.0.0.1', 1234))  # leaga socket-ul la IP si port

print("serverul asteapta mesaje...")

while True:
    input_data, addr = server_socket.recvfrom(1024)  # primeste date de la client
    print(f"date primite de la {addr}: {input_data.decode()}")

    response = f"echo: {input_data.decode()}"  # pregateste raspunsul
    server_socket.sendto(response.encode(), addr)  # trimite raspunsul inapoi
clientului
```

Server UDP - concurent

```
import socket
import threading

# functie pentru a gestiona fiecare mesaj de client
def handle_client(data, addr):
    print(f"date primite de la {addr}: {data.decode()}")
    response = f"echo: {data.decode()}"  # pregateste raspunsul
    server_socket.sendto(response.encode(), addr)  # trimite raspunsul inapoi
clientului

# configurarea serverului
server_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)  # creeaza un socket
UDP
server_socket.bind(('127.0.0.1', 1234))  # leaga socket-ul la IP si port

print("serverul asteapta mesaje...")

while True:
    input_data, addr = server_socket.recvfrom(1024)  # primeste date de la client
    client_thread = threading.Thread(target=handle_client, args=(input_data, addr))  #
creeaza un fir pentru client
```

```
    client_thread.start()  # porneste firul
```

Client UDP

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>
#include <arpa/inet.h>

int main() {
    int sock;
    struct sockaddr_in server;
    char message[100], server_reply[200];

    // creeaza socket
    sock = socket(AF_INET, SOCK_DGRAM, 0);
    if (sock == -1) {
        printf("eroare la crearea socketului\n");
        return 1;
    }

    server.sin_addr.s_addr = inet_addr("127.0.0.1");  // setez IP
    server.sin_family = AF_INET;  // setez tipul de adresa
    server.sin_port = htons(1234);  // setez portul

    printf("introduceti mesajul: ");
    scanf("%s", message);  // citeste mesajul de la utilizator

    // trimite mesajul
    sendto(sock, message, strlen(message), 0, (struct sockaddr*)&server,
sizeof(server));

    // primeste raspunsul
    socklen_t server_len = sizeof(server);
    recvfrom(sock, server_reply, sizeof(server_reply), 0, (struct sockaddr*)&server,
&server_len);
    printf("raspunsul de la server: %s\n", server_reply);

    close(sock);  // inchide socketul
    return 0;
}
```

Curs 5
Adrese. Adrese de Broadcast. LAN

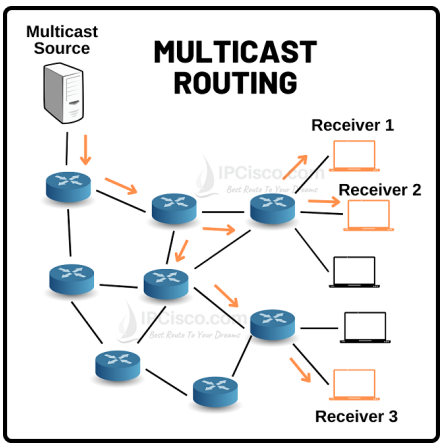
Adresă de broadcast = ultima adresă IP dintr-o clasă de adrese
- ↳ nu se poate seta manual
- ↳ tot ce se trimite pe adresa de broadcast ajunge la toți din rețea

→ se folosește UDP ca să se facă broadcast

Multicast

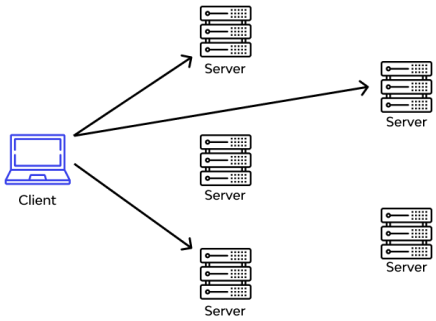
1 : m
- 1 trimite
- mai mulți primesc

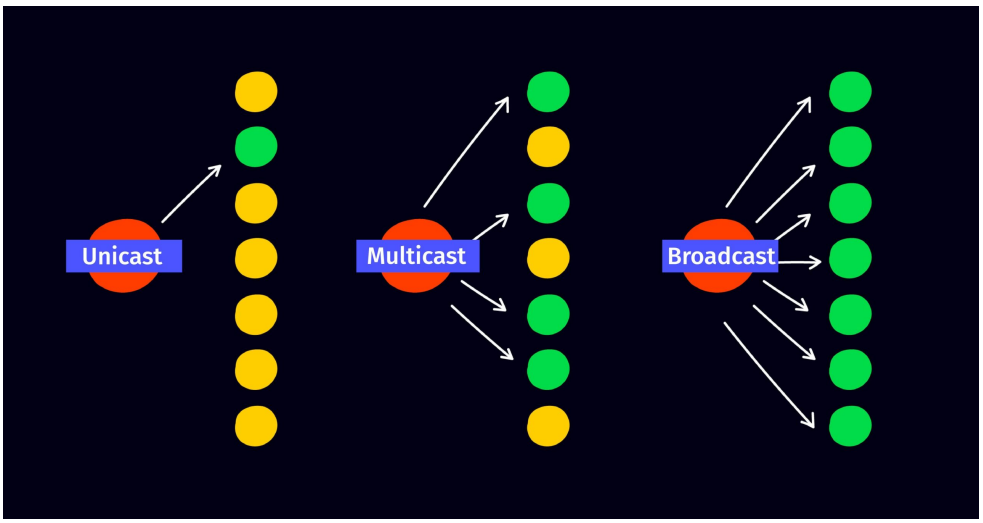
→ nu neapărat în aceași rețea locală
→ e la latitudinea receptorului dacă primește sau nu



Unicast

→ unul transmite, cineva primește (nu neapărat toți)

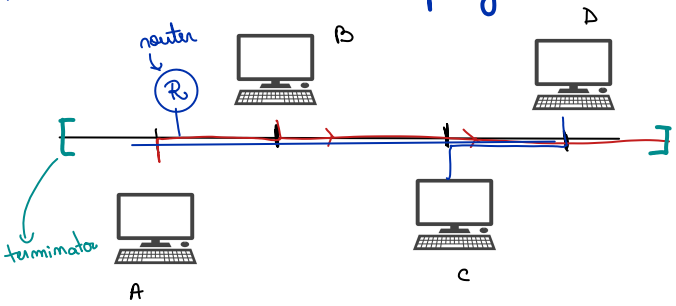




Topologii de rețea

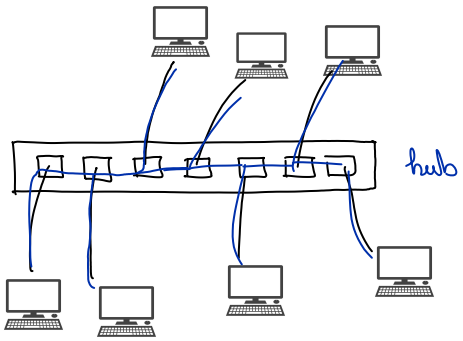
- LAN (Local Area Network) ⟶ VLAN (Virtual Local Area Network)
- MAN (Metropolitan Area Network) (ex) UBB
- WAN (Wide Area Network) (ex) rețea DIGI, tot internetul

1) BUS Network - Topologie liniară - BNC

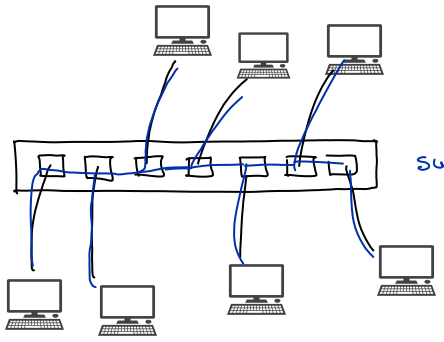


→ ce transmitea A spre B se vedea pe toată rețeaua
→ 10 Mbp
↳ problema coliziunilor
→ dacă se rupea rețeaua (firul) → fail

2) Star Network - Topologie stea

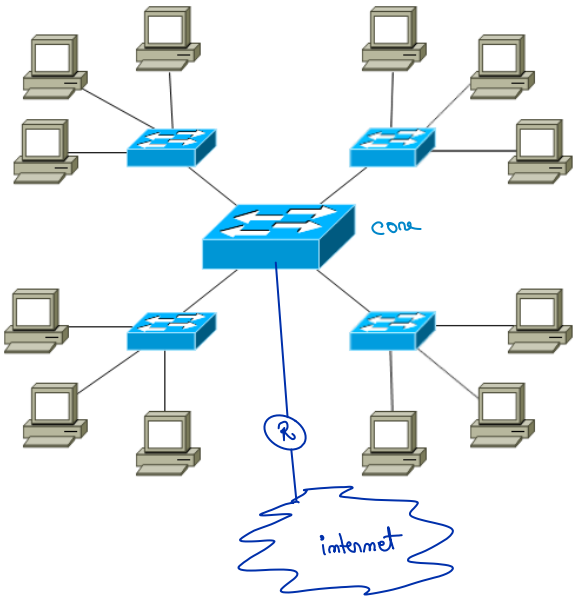


→ era mai fiabilă



switch ⟶ mai deștept decât un hub
↳ la interconectează fizic, dar și la nivelul de date
↳ ține minte adresele MAC

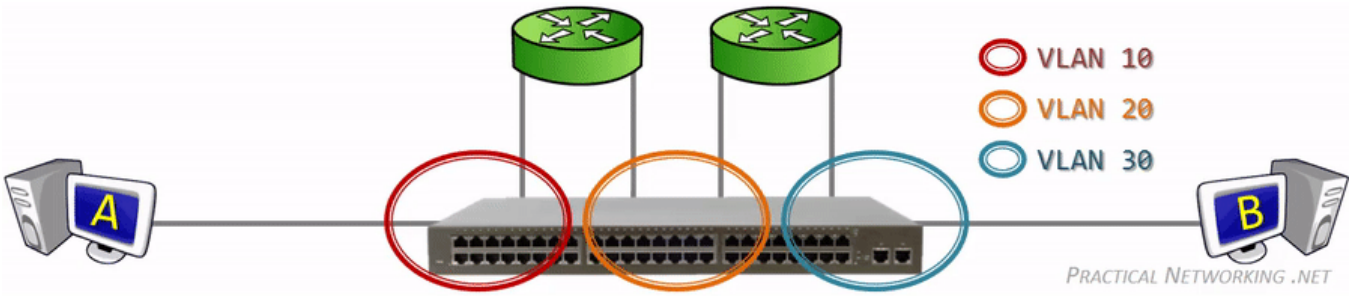
3) Extended Star Topology - Topologie Stea Extinsă



- are un core switch la care sunt conectate mai multe switch-uri
- port de uplink = portul care face legătura între switch-urile adiacente și cel central

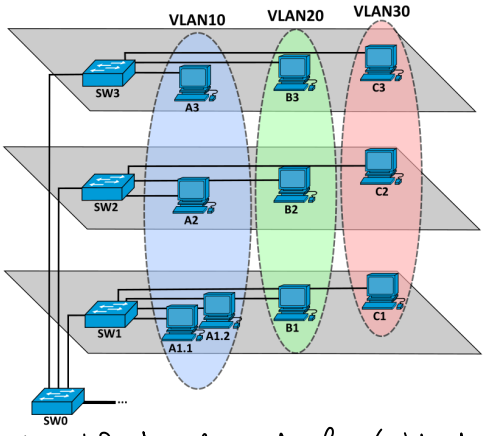
↳ în așa fel traficul generat de un VLAN nu poate fi vizibil în cealaltă rețea

VLAN



→ împarte o rețea fizică în mai multe rețele virtuale

→ dispozitivele din același VLAN pot comunica între ele, chiar dacă sunt conectate la switch-uri diferite, iar dispozitivele din VLAN-uri diferite nu pot comunica direct fără un router sau Layer 3 switch



! se pot lega 2 calculatoare / device-uri între ele fără internet? da, la nivel fizic (cablu direct sau crossover)

→ cablu ethernet (8 fire): 4 perechi de fire

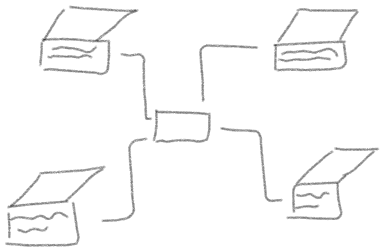
Protocol TFTP (Trivial File Transfer Protocol) → folosește UDP și este folosit în LAN-uri

Dacă într-o rețea avem mai mult de 2 device-uri (calculatoare) ai nevoie de un switch pentru a le conecta între ele.

TOPOLOGII

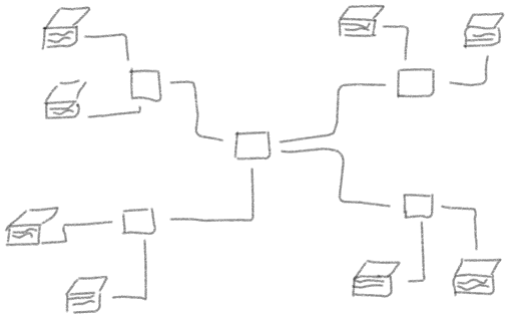
1 Star topology

↳ toate dispozitivele sunt conectate printr-un mod central (comutator / hub)
↳ dispozitivele sunt conectate în mod ind. prin modul central



2. Extended star topology

↳ se foloseste un dispozitiv central care e conectat la mai multe alte dispozitive

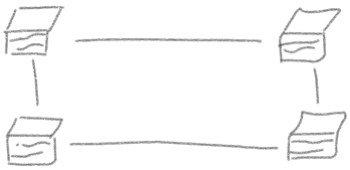


* 3 Bus Topology

↳ toate dispozitivele sunt conectate la un cablu comun
↳ datele de la un dispozitiv la altul sunt transmise prin magistrală și fiecare stație decide dacă să proceseze sau să ignore datele destinate altei stații

4 Ring Topology

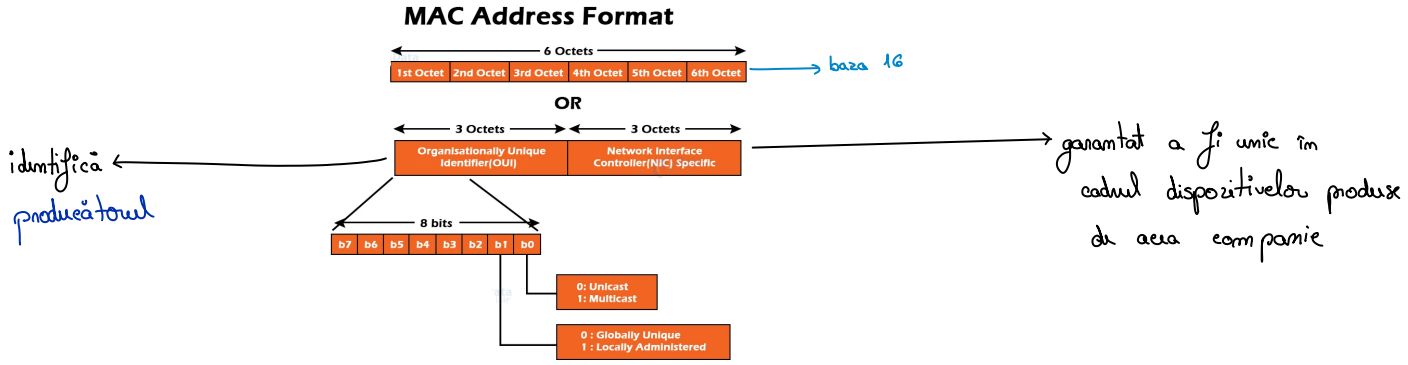
↳ datele circulă de la o stație la alta până ajunge la destinație



Adresă MAC / Ethernet / Fizică / Hardware

= identificator unic atribuit fiecărui dispozitiv de rețea, cum ar fi plăcile de rețea (NIC – Network Interface Card) din calculatoare, smartphone-uri, routere și alte echipamente de rețea

– folosit pt a gestiona comunicarea între dispozitive dintr-un LAN



– folosite de switch-uri pt a învăța topologia rețelei și a direcționa cadrele (frame-urile) către dispozitivele corecte

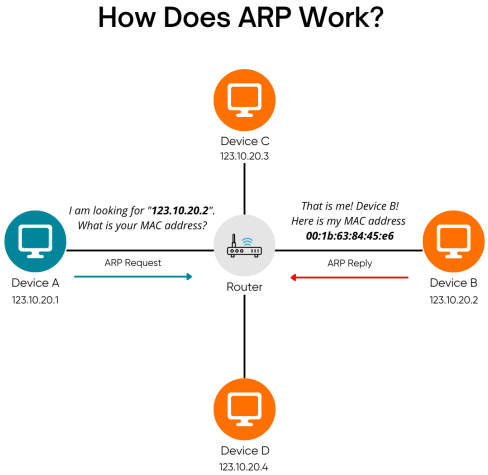
Tipuri de adrese MAC

1. **Unicast**: Adresa MAC este asociată unui singur dispozitiv. Cadrele de date sunt trimise doar către acel dispozitiv.
2. **Multicast**: Adresa MAC este asociată unui grup de dispozitive. Cadrele de date sunt trimise către toate dispozitivele din grup.
3. **Broadcast**: Adresa MAC `FF:FF:FF:FF:FF:FF` este folosită pentru a trimite cadre de date către toate dispozitivele din rețea.

→ hard codată în hardware-ul dispozitivului, dar poate fi modificată (spoofing) uneori

→ ⚠ este folosită DOAR în rețelele locale

- la nivel Rețea:
  - protocolul ARP – Address Resolution Protocol
    → găsește adresa MAC a unei adrese IP cunoscute
  - protocolul RARP – Reverse Address Resolution Protocol
    → găsește adresa IP a dispozitivului când o știe doar pe cea MAC (own device)



Exemplu de utilizare:

- Un dispozitiv fără IP (de exemplu, un terminal vechi) își cunoaște adresa MAC (`00:1A:2B:3C:4D:5E`) și trimite un RARP Request. Serverul RARP răspunde cu adresa IP `192.168.1.30`.

→ se trimite pe adresa de broadcast cererea

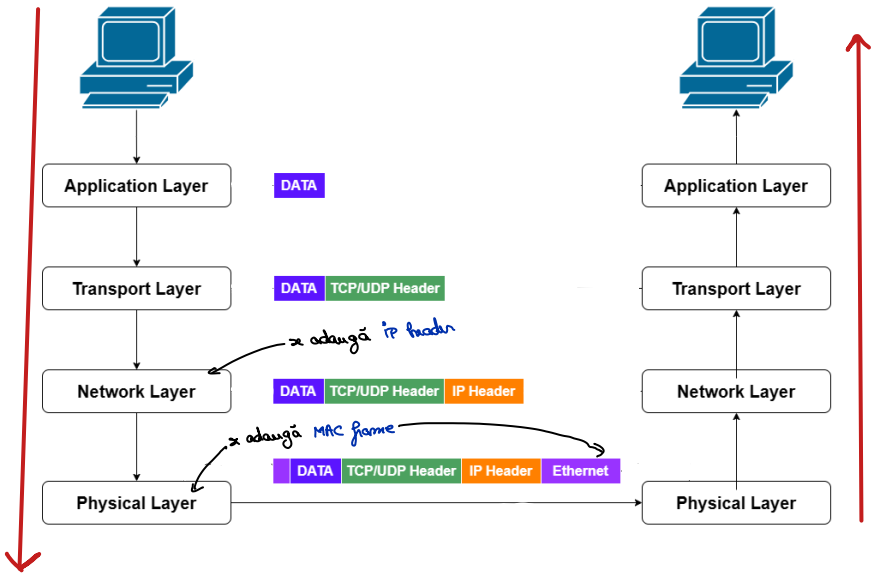
→ cu protocoalele se țin minte perechi

arp -n

Configurarea unui calculator pentru acces la internet

1) Adresă IP ⟶ unică în rețea
2) Mască de rețea – Netmask ⟶ determină care parte a adresei IP reprezintă rețeaua și care parte reprezintă dispozitivul
3) Default Gateway ⟶ adresa de rețea a dispozitivului (de obicei router) care face legătura între rețeaua locală și internet
   ⟶ toate cererile sunt trimise către acest gateway care le direcționează mai departe
   ⟶ e în aceeași rețea cu dispozitivul
4) DNS (Domain Name Server) ⟶ traduce numele de domenii în adrese IP
   (ex) 8.8.8.8 (Google DNS), 1.1.1.1 (Cloudflare DNS)

Curs 7



header IP:
- adresă IP sursă
- adresă IP destinație
- (alte info. necesare pt rutare)

frame MAC / Ethernet:
- adresa MAC sursă
- adresa MAC destinație (adresa fizică a dispozitiv. sau a gateway-ului)
- alte info. necesare

→ se pot afla adresele MAC doar ale dispozitivelor din aceeași rețea

- putem avea o rețea cu calculatoare care să comunice între ele fără să fie conectată la internet cât timp avem switch / hub

Curs 8

ICMP – Internet Control Message Protocol

↳ între nivel rețea și nivel transport

`arping -i eth0 [IP]`

`arp -n`

DHCP - Dynamic Host Configuration Protocol

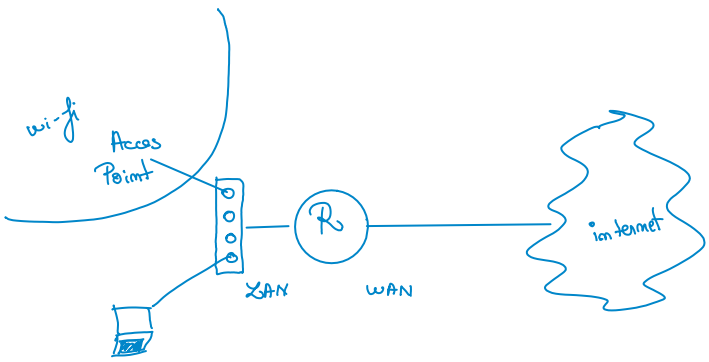
⟶ atribuie automat adrese IP și alte setări de rețea dispozitivelor dintr-o rețea

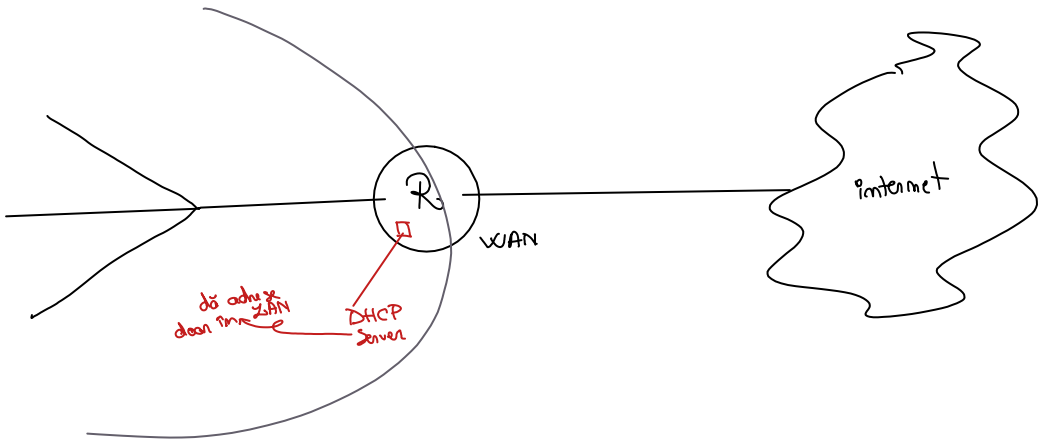
Procesul DHCP : DORA

1) Discover : clientul trimite un mesaj DHCP Discover (în mod broadcast) pt a găsi un server DHCP disponibil în rețea
2) Offer : serverul DHCP răspunde cu un mesaj DHCP Offer, oferind o adresă IP disp. & setările de rețea
3) Request: clientul acceptă oferta și trimite un mesaj DHCP Request pt a confirma folosirea IP-ului
4) Acknowledge: serverul confirmă alocarea adresei IP cu un mesaj DHCP Acknowledge ⟹ clientul configurează acum adresa IP

— router = DHCP server uneori, dar și DNS server

— dacă avem 2 servere DHCP ⟹ conflict în rețea și se poate întâmpla să se dea aceleași adrese IP





Subnetting

Netmask = spune câți octeți caracterizează rețeaua
- formată din biți → 1: câți biți sunt comuni
  → 0: biții diferiți a IP-ului device-ului

- **Adresă de rețea:** adresa IP care identifică în mod unic o rețea (întotdeauna nr par) → prima din rețea
- **Adresă de broadcast:** adresa IP pe care se poate transmite în toată rețeaua (întotdeauna nr impar) → ultima

tot internetul: 0.0.0.0 → 255.255.255.255 → 2^{32} adrese IP
Netmask: 0.0.0.0

Netmasks ne împart internetul în subrețele

! Nu putem spune că o adresă IP e o adresă de rețea fără netmask-ul ei

Adresă de rețea: NA = IP AND NM
Adresă de broadcast: BA = IP OR (NOT NM)

IP public: - adresă unică la nivel global, atribuită unui dispozitiv sau unei rețele pt a permite comunicarea pe internet
- folosită pt a identifica dispozitivele și rețelele în exteriorul LAN-ului
- utilizare: accesare site-uri web, permite conexiuni externe

IP privat: - adresă utilizată într-o LAN pt a permite comunicarea între dispozitivele din aceeași rețea
- atribuită automat de un router sau configurare manuală
- nu e unică ∀

ex) 172.16.0.0 → 172.31.255.255 → /12
192.168.0.0 → 192.168.255.255 → /16

- utilizare: pt a permite dispozitivelor să partajeze o singură conexiune la internet prin intermediul unui router (folosind **NAT - Network Address Translation**)

CIDR	Subnet mask	Wildcard mask	# of IP addresses	# of usable IP addresses
/32	255.255.255.255	0.0.0.0	1	1
/31	255.255.255.254	0.0.0.1	2	2*
/30	255.255.255.252	0.0.0.3	4	2
/29	255.255.255.248	0.0.0.7	8	6
/28	255.255.255.240	0.0.0.15	16	14
/27	255.255.255.224	0.0.0.31	32	30
/26	255.255.255.192	0.0.0.63	64	62
/25	255.255.255.128	0.0.0.127	128	126
/24	255.255.255.0	0.0.0.255	256	254
/23	255.255.254.0	0.0.1.255	512	510
/22	255.255.252.0	0.0.3.255	1,024	1,022

Curs 9

Clase de adrese

RFC: Request for Comments, este o serie de documente tehnice utilizate pt a descrie standarde, protocoale, proceduri etc pt rețele de calculatoare

Class	First Octet decimal (range)	First Octet binary (range)	IP range	Subnet Mask	Hosts per Network ID	# of networks
Class A	0 – 127	0XXXXXXX	0.0.0.0-127.255.255.255	255.0.0.0	$2^{24}-2$	$2^7 = 128$
Class B	128 – 191	10XXXXXX	128.0.0.0-191.255.255.255	255.255.0.0	$2^{16}-2$	2^{14}
Class C	192 – 223	110XXXXX	192.0.0.0-223.255.255.255	255.255.255.0	2^8-2	2^{21}
Class D (Multicast)	224 – 239	1110XXXX	224.0.0.0-239.255.255.255			
Class E (Experimental)	240 – 255	1111XXXX	240.0.0.0-255.255.255.255			

→ 0.x.y.z → nu se folosește în general

→ definite pe baza primilor biți ai IP-ului și a nr de biți alocați pt rețea și host

→ adresa 127.x.x.x este rezervată pt testare locală (loopback) → local host

→ adresele de rețea și broadcast sunt rezervate

Adrese private (non-rutabile pe internet)

Anumite intervale de adrese sunt rezervate pentru **rețele private** (LAN) și nu pot fi utilizate pe internet:

- Clasa A → 10.0.0.0 – 10.255.255.255 /8
- Clasa B → 172.16.0.0 – 172.31.255.255 /12
- Clasa C → 192.168.0.0 – 192.168.255.255 /16

Adresă de rețea: NA = IP AND NM

Adresă de broadcast: BA = IP OR (NOT NM)

(ex)

IP: 192.168. 1. 117 &&
NM: 255.255.255. 0 → /24 (24 de biți 1)
NA: 192.168. 1. 0

IP: 192.168. 1. 117 ||
NM: 0. 0. 0.255
BA: 192.168. 1.255

Operații pe biți

a	b	a&b	a|b	a^b	~a
0	0	0	0	0	1
0	1	0	1	1	1
1	0	0	1	1	0
1	1	1	1	0	0

172.30.x.y

⇒ 172.30.0.0 → 172.30.255.255

NM: 255.255.0.0 → /16 ↪ 2^{16} adrese IP

(ex) 192.168.1.0 / 255.255.255.0

192.168.1.0/24

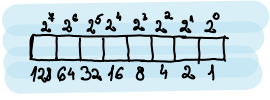
Network Address | Netmask

172.30.0.0 /255.255.0.0

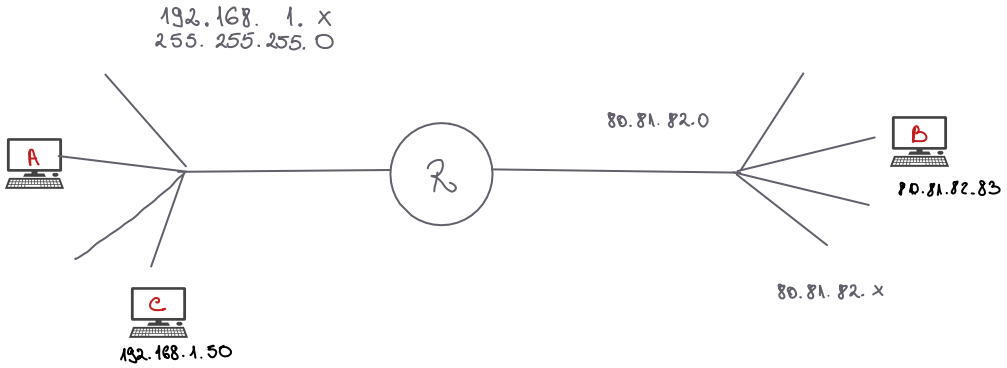
172.30.0.0/16

(ex) 10.x.y.z

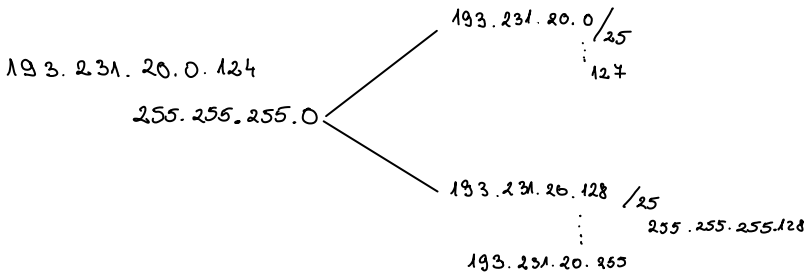
255.0.0.0/8 ↪ NA 10.0.0.0 → BA 10.255.255.255 → 2^{24}

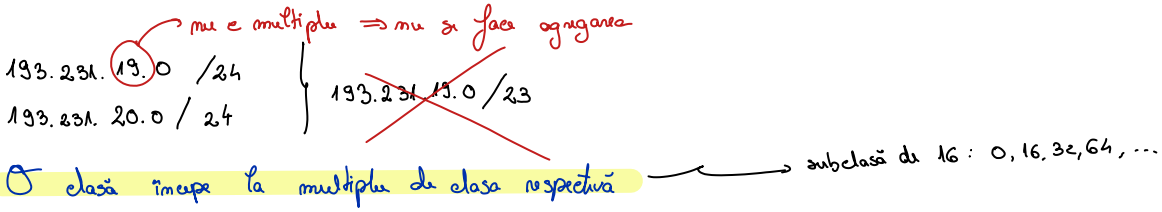


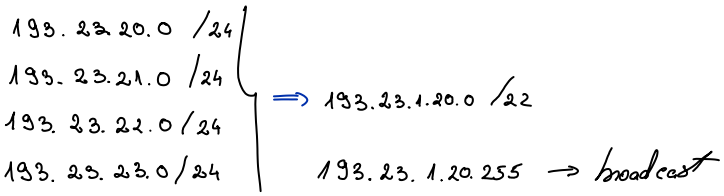
Internet : 0.0.0.0 /0.0.0.0



A → C verif dacă e în aceeași rețea, face ARP request
tot ce e la nivelul rețelei locale rămâne la nivelul rețelei locale







Spoofing și Sniffing în Securitatea Rețelelor

În securitatea rețelelor, **spoofing** și **sniffing** sunt două tehnici utilizate de atacatori pentru a compromite integritatea, confidențialitatea și disponibilitatea datelor. Aceste atacuri pot duce la interceptarea, modificarea sau furtul de informații critice.

1. Spoofing

Spoofing (înșelăciune digitală) este o tehnică prin care un atacator își falsifică identitatea pentru a părea o sursă de încredere. Acest tip de atac este folosit pentru a ocoli mecanismele de autentificare, a intercepta comunicații sau a lansa atacuri mai complexe, cum ar fi **Man-in-the-Middle (MitM)** sau **Denial-of-Service (DoS)**.

1.1. Tipuri de spoofing

1. **IP Spoofing**
   - Atacatorul falsifică adresa IP a unui dispozitiv legitim pentru a părea că este un utilizator autentic.
   - Folosit frecvent în atacurile **DDoS**, unde pachetele sunt trimise de la adrese false pentru a supraîncărca serverele.
   - Exemple: Atacurile **SYN Flood** sau **Smurf Attack**.
2. **ARP Spoofing (ARP Poisoning)**
   - Atacatorul modifică tabelele ARP dintr-o rețea locală (LAN), asociind adresa sa MAC cu adresa IP a unui dispozitiv de încredere (ex: gateway).
   - Astfel, traficul este redirecționat către atacator, permițându-i să intercepteze, modifice sau blocheze datele.
   - Utilizat adesea în atacuri **Man-in-the-Middle (MitM)**.
3. **DNS Spoofing (DNS Cache Poisoning)**
   - Atacatorul introduce informații false în memoria cache a unui server DNS, determinând utilizatorii să fie direcționați către site-uri frauduloase.
   - Poate duce la **furt de date bancare**, **infectare cu malware** sau **phishing**.
4. **Email Spoofing**
   - Atacatorul trimite emailuri cu o adresă de expeditor falsificată, făcând mesajul să pară autentic.
   - Utilizat în atacurile **phishing** pentru a convinge victimele să divulge informații sensibile.
5. **Caller ID Spoofing**
   - Falsificarea numărului de telefon afișat la apeluri pentru a păcăli victima să răspundă.
   - Folosit în fraude bancare sau escrocherii telefonice.

1.2. Măsuri de protecție împotriva spoofing-ului

- **Filtrare IP și ACLs (Access Control Lists)** – Blochează pachetele suspecte care provin din surse nerecunoscute.
- **Utilizarea protocoalelor de securitate** – **HTTPS, DNSSEC, SPF/DKIM/DMARC** pentru email.
- **ARP Inspection și Port Security** – Configurarea switch-urilor pentru a preveni atacurile ARP Spoofing.
- **Autentificare multifactor (MFA)** – Protejează conturile împotriva atacurilor de phishing.

2. Sniffing

Sniffing este procesul de interceptare și analizare a traficului de rețea. Poate fi utilizat în scopuri legitime (depanare, monitorizare), dar și malițioase, pentru a captura informații sensibile precum parole, date bancare sau mesaje private.

2.1. Tipuri de sniffing

1. **Passive Sniffing**
   - Atacatorul ascultă pasiv traficul într-o rețea și colectează date fără a interacționa cu ele.
   - Funcționează în rețele **hub-based**, unde tot traficul este transmis către toate dispozitivele.
   - Exemplu: Capturarea de date cu **Wireshark**.
2. **Active Sniffing**
   - Atacatorul manipulează traficul pentru a intercepta date, utilizând tehnici precum **ARP Spoofing**, **MAC Flooding** sau **DNS Spoofing**.
   - Funcționează în rețele **switch-based**, unde datele sunt direcționate doar către dispozitivele destinate.
3. **Man-in-the-Middle (MitM)**
   - Atacatorul se interpune între două părți care comunică, interceptând, modificând sau redirecționând datele fără ca acestea să știe.
   - Poate fi realizat prin **HTTPS Stripping**, **Wi-Fi Fake AP** sau **Session Hijacking**.
4. **Packet Injection**
   - Atacatorul modifică sau introduce pachete în traficul rețelei pentru a manipula comunicațiile.
   - Folosit în atacuri precum **DNS Hijacking** sau **Rogue DHCP Server**.

2.2. Instrumente utilizate în sniffing

- **Legitime**:
  - **Wireshark** – Monitorizare și analiză de pachete.
  - **Tcpdump** – Captură de trafic din linia de comandă.
- **Malițioase**:
  - **Ettercap** – Atacuri MitM și ARP Spoofing.
  - **dsniff** – Capturare de parole și date sensibile.
  - **Cain & Abel** – Recuperare de parole și sniffing.

5. 2.3. Măsuri de protecție împotriva sniffing-ului

- **Utilizarea criptării end-to-end** (SSL/TLS, VPN) pentru a proteja datele în tranzit.
- **Implementarea rețelelor switched** pentru a limita accesul la pachete neautorizate.
- **Utilizarea de certificate digitale** pentru a preveni atacurile MitM.
- **Monitorizarea rețelei** și utilizarea de IDS/IPS pentru detectarea activităților suspecte.

Agregarea Rețelelor (Supernetting, Route Aggregation)

Agregarea rețelelor este procesul de combinare a mai multor rețele IP mai mici într-o rețea mai mare, reducând astfel numărul de rute dintr-o tabelă de rutare. Aceasta este folosită pentru a îmbunătăți eficiența rutării și a reduce dimensiunea tabelelor de rutare în rețelele mari.

1. Ce este Supernetting (Route Aggregation)?

Supernetting implică **gruparea mai multor rețele IP contigue într-o singură intrare de rutare**, reducând numărul de rute anunțate de un router.

🔹 **Exemplu simplu:**
Avem patru rețele de clasă C:

- **192.168.1.0/24**
- **192.168.2.0/24**
- **192.168.3.0/24**
- **192.168.4.0/24**

În loc să avem 4 rute separate, putem face agregare într-o singură rută:

- **192.168.0.0/22 (mască 255.255.252.0)**

Acest lucru reduce complexitatea rețelei și optimizează utilizarea adreselor IP.

2. Avantajele agregării de rețele

✅ **Reducerea dimensiunii tabelelor de rutare** – Routerele gestionează mai puține rute individuale.
✅ **Eficiență crescută a rutării** – Mai puține actualizări de rută și un consum mai mic de resurse.
✅ **Reducerea consumului de memorie și procesare** pe routere.
✅ **Îmbunătățirea convergenței** în protocoale de rutare (OSPF, BGP, EIGRP).

3. Cum se face agregarea de rețele?

Pas 1: Identificarea rețelelor contigue

- Rețelele trebuie să fie **adiacente** (ex: 192.168.1.0/24 și 192.168.2.0/24).
- Trebuie să aibă o **mască comună mai mare** pentru a fi agregate.

Pas 2: Alegerea unei măști de rețea mai mari

Folosim **CIDR (Classless Inter-Domain Routing)** pentru a determina cea mai mică mască comună care acoperă toate rețelele.

🔹 **Exemplu:**
Avem rețelele **10.1.0.0/24** și **10.1.1.0/24**.

Le putem agrega într-o singură rețea **10.1.0.0/23** (mască **255.255.254.0**).

Pas 3: Anunțarea agregării

- În **OSPF** – folosim **summarization** la nivel de Area Border Router (ABR).
- În **BGP** – utilizăm agregare de rute pentru a optimiza anunțurile către alte rețele.

4. Supernetting vs Subnetting

Caracteristică	Subnetting	Supernetting
Scop	Împărțirea unei rețele mari în subrețele mai mici	Combinarea mai multor rețele mai mici într-una mai mare
Crește sau reduce numărul de rețele?	Crește numărul de rețele	Reduce numărul de rețele
Masca de subrețea	Crește (ex: /24 → /26)	Scade (ex: /24 → /22)
Utilizare	Rețele LAN, VLAN-uri	Internet, rutare BGP

5. Exemple practice

🔹 **Exemplu cu 4 rețele /24**

Rețele:

- 172.16.0.0/24
- 172.16.1.0/24
- 172.16.2.0/24
- 172.16.3.0/24

Le agregăm într-o singură rețea:

- **172.16.0.0/22** (mască **255.255.252.0**)

🔹 **Exemplu cu BGP (Border Gateway Protocol)**

Dacă un ISP gestionează multiple rețele, în loc să anunțe:

- **203.0.113.0/24**
- **203.0.114.0/24**

Poate agrega într-un singur prefix:

- **203.0.112.0/22**

193.231.20.0 / 255.255.255.0 → 256 adrese IP în fiecare
193.231.21.0 / 255.255.255.0

193.231.20.0 / 255.255.254.0 ⟺ /23 → 512 adrese IP

Port #	Application Layer Protocol	Type	Description
20	FTP	TCP	File Transfer Protocol - data
21	FTP	TCP	File Transfer Protocol - control
22	SSH	TCP/UDP	Secure Shell for secure login
23	Telnet	TCP	Unencrypted login
25	SMTP	TCP	Simple Mail Transfer Protocol
53	DNS	TCP/UDP	Domain Name Server
67/68	DHCP	UDP	Dynamic Host
80	HTTP	TCP	HyperText Transfer Protocol
123	NTP	UDP	Network Time Protocol
161,162	SNMP	TCP/UDP	Simple Network Management Protocol
389	LDAP	TCP/UDP	Lightweight Directory Authentication Protocol
443	HTTPS	TCP/UDP	HTTP with Secure Socket Layer

DNS = 53
HTTP = 80

SNAT (Source Network Address Translation)

Ce este SNAT?

SNAT (Source Network Address Translation) este un mecanism de **modificare a adresei IP sursă** a unui pachet care pleacă dintr-o rețea. Este utilizat în principal pentru a permite dispozitivelor cu adrese IP private să acceseze internetul folosind o adresă IP publică.

Cum funcționează SNAT?

1. **Un dispozitiv dintr-o rețea locală trimite un pachet către o destinație externă** (ex. internet).
2. **Routerul/NAT Gateway-ul înlocuiește adresa IP sursă a pachetului** cu o altă adresă IP (de obicei publică).
3. **Pachetul modificat este trimis către destinație**, iar serverul răspunde către noua adresă IP sursă.
4. **Routerul face maparea inversă** și redirecționează răspunsul către dispozitivul inițial.

Ce tipuri de adrese poate translata SNAT?

SNAT poate translata adrese **false (private) și reale (publice)**, în diverse combinații:

Tip de translație	Explicație	Exemplu
✅ **Privat → Public** (fals → real)	Cel mai folosit caz: dispozitive cu IP privat accesează internetul folosind un IP public.	`192.168.1.10 → 203.0.113.5`
✅ **Public → Public** (real → real)	Se folosește în load balancing sau în cloud, unde un IP public este înlocuit cu alt IP public.	`198.51.100.10 → 203.0.113.20`
✅ **Privat → Privat** (fals → fals)	Rar folosit, dar poate apărea în rețele interne mari, unde un grup de adrese private este translatat în alt grup de adrese private.	`192.168.1.10 → 10.0.0.5`
❌ **Public → Privat** (real → fals)	**SNAT nu poate face acest lucru**, dar **DNAT (Destination NAT)** poate redirecționa traficul de la o adresă publică către una privată (ex. Port Forwarding).	`203.0.113.10 ❌→ 192.168.1.100`

Dacă avem nevoie de Public → Privat? → DNAT

Dacă vrem să modificăm **destinația** unui pachet astfel încât să ajungă la un IP privat, se folosește **DNAT (Destination NAT)**, nu SNAT.

Exemplu DNAT (Public → Privat):

- Un server public (`203.0.113.10`) primește o cerere pe portul 80.
- Routerul schimbă adresa **destinație** către un server privat (`192.168.1.100`).
- Acesta răspunde înapoi, iar routerul face maparea inversă pentru ca pachetul să pară că vine de la IP-ul public.

💡 **Acest proces este folosit în Port Forwarding și NAT Hairpinning.**

Unde este folosit SNAT?

✅ **Acces la internet pentru dispozitive cu IP privat** (cel mai frecvent caz)
✅ **Load Balancing și Failover în cloud** (pentru distribuirea traficului)
✅ **Securitate și filtrare** (ascunderea adreselor reale ale clienților)
✅ **Rețele mari interne** (unde un grup de IP-uri private este convertit în alt grup)

Exemplu de SNAT pe Linux (iptables)

```
iptables -t nat -A POSTROUTING -o eth0 -j SNAT --to-source 203.0.113.5
```

👉 Această regulă modifică **IP-ul sursă** al tuturor pachetelor care ies prin interfața **eth0**, înlocuindu-l cu **203.0.113.5**.

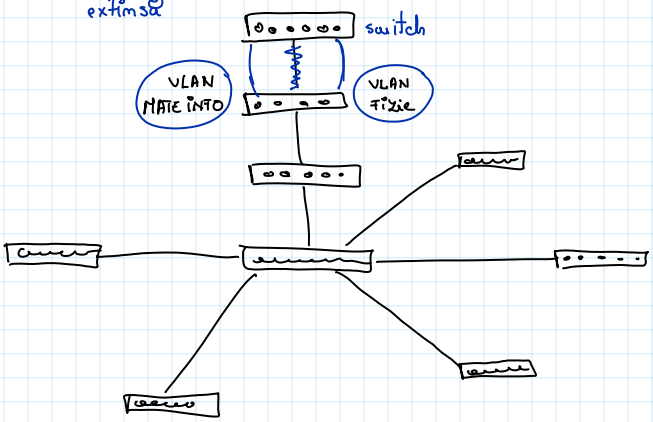
Concluzie:

✅ **SNAT:** Poate face **fals → real**, **real → real**, **fals → fals**.
❌ **SNAT NU poate face real → fals**, dar **DNAT poate redirecționa pachete de la un IP real spre un IP fals**.

Dacă ai nevoie de mai multe explicații sau exemple, întreabă! 🚀

Curs 10
VLAN

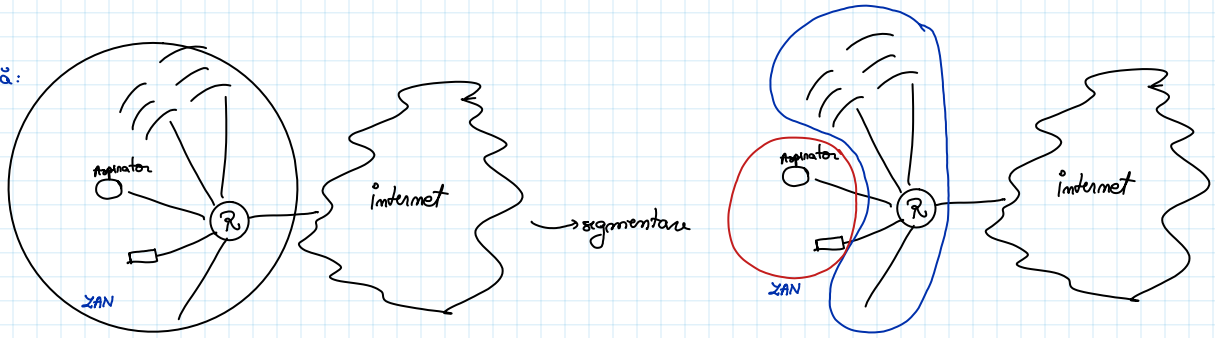
topologie stea extinsă



VLAN → Data Link

Cum se rupe o rețea ajunsă la capacitate maximă ?
De ce fragmentăm rețelele ? → securitate

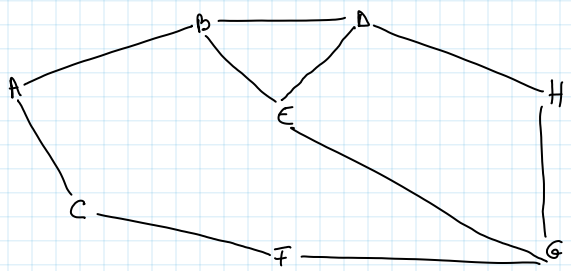
Acasă:



- la nivel local se poate face segmentare ⟶ pt securitate
- Guest Network de la router

Dirijare / Routare

Ruta



← uneori pt un drum mai lung dacă are viteză mai mare

Ⓐ

Destinație	Vecin	Drum de un cost
Ⓐ	Ⓐ	0
Ⓑ	Ⓑ	1
Ⓒ	Ⓒ	1
Ⓓ	Ⓑ	2
Ⓔ	Ⓑ	2
Ⓕ	Ⓒ	2
Ⓖ	Ⓒ	3
Ⓗ	Ⓑ	2

← tabelă de dirijare

← rută = înregistrare din tabel

Algoritmi de dirijare

- dirijare statică
- dirijare dinamică → algoritmi de dirijare bazați pe vectori distanță (Bellman-Ford)
  → algoritmi de dirijare bazați pe starea legăturilor
- dirijare prin inundare

1) Vectori distanță

A			B			C			D		
Dest.	Vecin	Cost									
A	A	0	B	B	0	C	C	0	D	D	0
B	B	1+0	A	A	0+1	A	A	0+1			
						B	A	1+1			

2) Starea legăturilor (adiacent routerului) → sunt trimise prin inundație

Ⓐ

(A, B, 1, timestamp)

(A, C, 1, timestamp)

de completat

dirijare dinamică → algoritmi de dirijare bazați pe vectori distanță (Bellman-Ford)
→ algoritmi de dirizare bazați pe starea legăturilor (Dijkstra)

Algoritmi de dirijare

1. Algoritmi de tip Vectori de Distanță (Distance Vector - DV)

problema numărării la infinit

Acești algoritmi funcționează pe principiul că fiecare router menține o **tabelă de rutare** care conține:

- Destinația (alte routere din rețea).
- Distanța către fiecare destinație (exprimată în număr de hop-uri sau altă metrică).
- Cel mai bun **next-hop** (router vecin prin care se ajunge la destinație).

Cum funcționează?

1. **Inițializare:** Fiecare router își cunoaște doar vecinii direcți.
2. **Actualizare periodică:** Routerele trimit tabelele lor de rutare către vecini la intervale regulate.
3. **Regula Bellman-Ford:** Dacă un router primește o informație mai bună (o distanță mai mică) despre o destinație de la un vecin, își actualizează tabela.
4. **Propagare lentă a schimbărilor:** Dacă apare o modificare în rețea (de exemplu, cade un link), aceasta trebuie propagată pas cu pas prin toate routerele.

Avantaje și Dezavantaje:

✅ **Avantaje:**

- Ușor de implementat și de configurat.
- Consum redus de resurse.

❌ **Dezavantaje:**

- **Convergență lentă** (durează până când toate routerele învață noua configurație).
- **Problema buclelor** – dacă un link cade, routerele pot intra într-o buclă în care trimit pachetele înainte și înapoi (exemplu: „contorizare la infinit”).
- **Limitări în rețele mari** – majoritatea implementărilor (ex. RIP) nu suportă mai mult de 15 hop-uri.

📌 **Exemplu de protocol bazat pe Distance Vector:**

- **RIP (Routing Information Protocol)** – folosește numărul de hop-uri ca metrică și trimite actualizări la fiecare 30 de secunde.

2. Algoritmi de tip Starea Legăturilor (Link State - LS)

Acești algoritmi adoptă o abordare diferită: în loc să păstreze doar informații despre vecini, fiecare router construiește o **hartă completă a rețelei** și calculează cea mai bună rută către fiecare destinație.

Cum funcționează?

1. **Descoperirea vecinilor:** Routerele trimit mesaje speciale pentru a detecta ce alte routere sunt conectate direct.
2. **Schimb de informații:** Fiecare router trimite un mesaj numit **LSA (Link State Advertisement)**, care conține toate informațiile despre conexiunile sale.

3. **Construirea topologiei:** Toate routerele colectează aceste mesaje și își construiesc propria hartă completă a rețelei.
4. **Algoritmul Dijkstra:** Routerele folosesc **Dijkstra (Shortest Path First - SPF)** pentru a calcula cea mai scurtă cale către fiecare destinație.

Avantaje și Dezavantaje:

✅ **Avantaje:**

- **Convergență rapidă** – schimbările din rețea sunt detectate și propagate rapid.
- **Fără bucle** – deoarece fiecare router are o imagine completă a rețelei, deciziile sunt mai precise.
- **Scalabilitate mai bună** în comparație cu Distance Vector.

❌ **Dezavantaje:**

- **Mai multă memorie și procesare necesare** – fiecare router trebuie să stocheze și să analizeze întreaga rețea.
- **Complexitate mai mare** în implementare și administrare.

📌 **Exemplu de protocol bazat pe Link State:**

- **OSPF (Open Shortest Path First)** – folosește costul link-urilor ca metrică și actualizează rutele doar când apare o schimbare.

Comparație rapidă

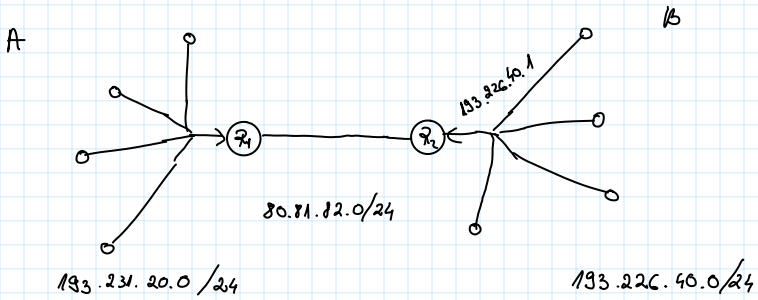
Caracteristică	Distance Vector (DV)	Link State (LS)
Cum funcționează	Routerele cunosc doar vecinii și propagă informația pas cu pas	Routerele construiesc o hartă completă a rețelei
Algoritm	Bellman-Ford	Dijkstra (SPF)
Exemplu de protocol	RIP	OSPF
Viteza de convergență	Lentă	Rapidă
Scalabilitate	Limitată	Bună
Consum de resurse	Redus	Mai mare
Probleme	Posibile bucle	Mai complex, consumă mai multe resurse

Pe scurt:

- **DV (Distance Vector)** → Routerele cunosc doar vecinii și schimbă informații periodic. Bun pentru rețele mici.
- **LS (Link State)** → Routerele știu toată rețeaua și calculează rutele optim. Bun pentru rețele mari.

Curs 11

! Nu orice router folosește un algoritm de dirijare



Tabelă pe R_1

Destinație	Vecin
193.226.40.0/24	R_2 (80.81.82.2)

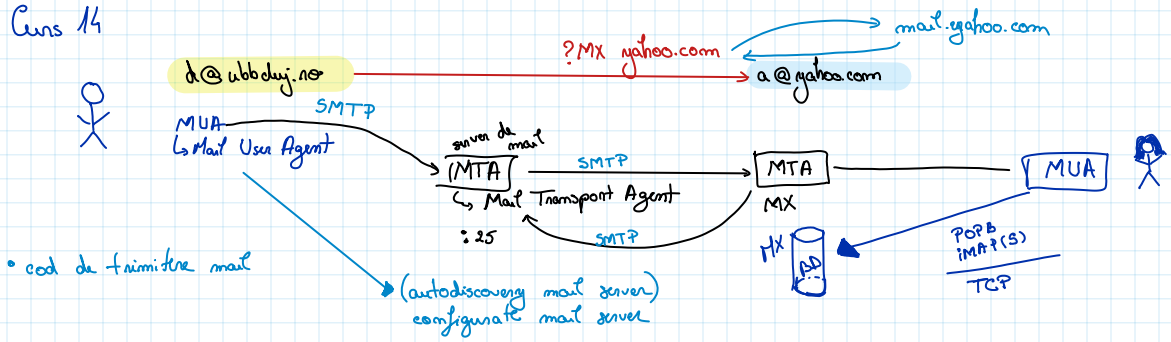
Tabelă pe R_2

Destinație	Vecin
193.231.20.0/24	R_1 (80.81.82.1)

Fiecare calculator are tabelă de dirijare:

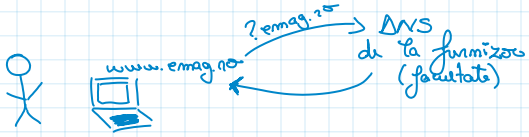
Windows: cmd ⇒ route print

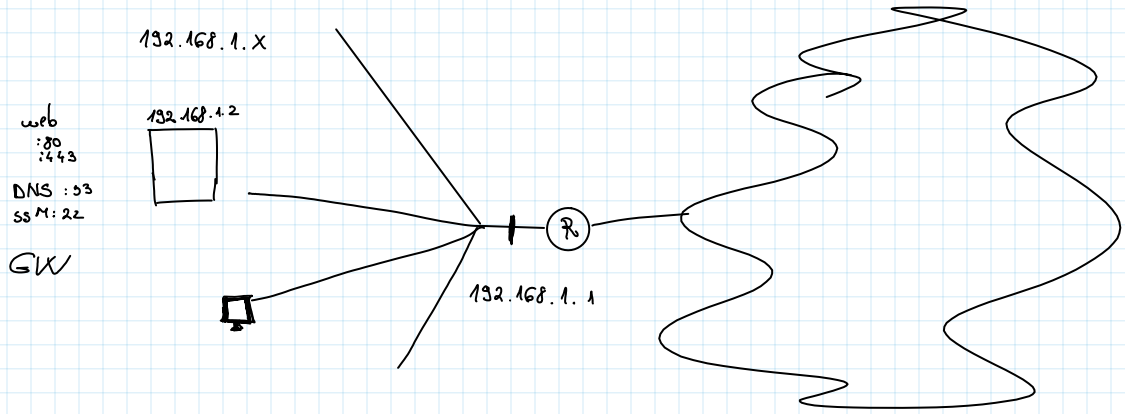
Curs 14



• cod de trimitere mail

Server Mail Open to Spam — Server Mail Railway





Tipuri de cabluri utilizate în rețelistică

În rețelistică, cablurile sunt utilizate pentru a conecta dispozitive precum computere, switch-uri, routere și alte echipamente de rețea. Cele mai comune tipuri de cabluri sunt **cablurile Ethernet**, care pot fi **directe (straight-through)** sau **încrucișate (crossover)**.

1. Cablul Direct (Straight-Through) → dispozitive diferite

🔹 **Descriere:**

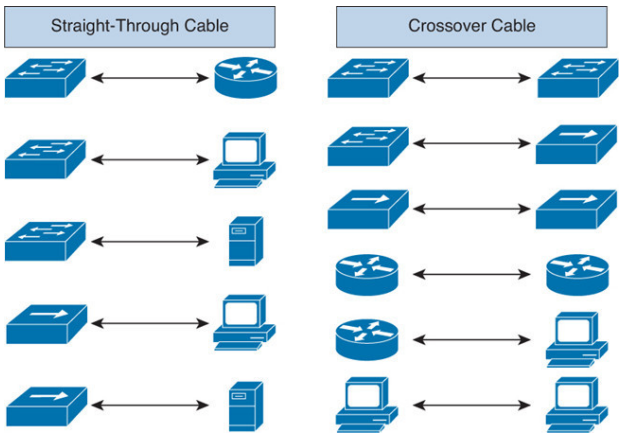
- Folosit pentru conectarea **dispozitivelor diferite** între ele.
- Pinii de pe un capăt al cablului sunt identici cu cei de pe celălalt capăt.
- Conectorii RJ-45 respectă standardele **TIA/EIA 568A – 568A** sau **568B – 568B**.

🔹 **Utilizare:**

✓ PC → Switch
✓ PC → Router
✓ Switch → Router
✓ Hub → Router

🔹 **Schema de cablare:** (TIA/EIA-568B pe ambele capete)

Pin	Culoare	Semnal
1	Portocaliu-alb	TX+
2	Portocaliu	TX-
3	Verde-alb	RX+
4	Albastru	Neutilizat
5	Albastru-alb	Neutilizat
6	Verde	RX-
7	Maro-alb	Neutilizat
8	Maro	Neutilizat



2. Cablul Încrucișat (Crossover) → dispozitive similare

🔹 **Descriere:**

- Folosit pentru conectarea **dispozitivelor similare** între ele.
- Pinii de transmisie (TX) și recepție (RX) sunt **încrucișați** la unul dintre capetele cablului.
- Un capăt folosește **TIA/EIA-568A**, iar celălalt **TIA/EIA-568B**.

🔹 **Utilizare:**

✓ PC → PC (direct, fără switch)
✓ Switch → Switch (dacă nu suportă Auto-MDIX)

✓ Hub → Hub
✓ Router → Router

🔹 **Schema de cablare:**
Capăt 1 (**TIA/EIA-568A**) → Capăt 2 (**TIA/EIA-568B**)

Pin (Capăt 1)	Culoare (568A)	Pin (Capăt 2)	Culoare (568B)
1	Verde-alb	1	Portocaliu-alb
2	Verde	2	Portocaliu
3	Portocaliu-alb	3	Verde-alb
6	Portocaliu	6	Verde

ℹ **Notă:** Celelalte fire nu sunt utilizate în rețele Fast Ethernet (10/100 Mbps), dar sunt necesare pentru Gigabit Ethernet.

3. Cablul Ethernet (Standard, Twisted Pair – UTP, STP, FTP)

📌 **Ethernet** este tehnologia dominantă pentru rețelele cablate. Cablurile Ethernet sunt clasificate în funcție de **categoria lor**:

- **Cat5e** – Suportă 1 Gbps (Gigabit Ethernet).
- **Cat6** – Suportă 10 Gbps pe distanțe scurte.
- **Cat7, Cat8** – Folosite pentru viteze mai mari și protecție mai bună.

📌 **Tipuri de cabluri Ethernet:**

Tip	Protecție	Utilizare
UTP (Unshielded Twisted Pair)	Fără protecție	Comun în rețele LAN casnice și birouri
STP (Shielded Twisted Pair)	Protecție pe fiecare pereche	Reduce interferențele electromagnetice
FTP (Foiled Twisted Pair)	Ecranare generală	Protejează împotriva perturbațiilor externe

4. Auto-MDIX: Eliminarea necesității cablurilor crossover

🔹 **Auto-MDIX (Automatic Medium-Dependent Interface Crossover)** este o tehnologie care permite dispozitivelor de rețea să detecteze și să ajusteze conexiunea automat.
🔹 **Dispozitive moderne (switch-uri, routere, plăci de rețea)** suportă Auto-MDIX, eliminând necesitatea cablurilor crossover.

📌 **Dispozitive care suportă Auto-MDIX:**
✓ Majoritatea switch-urilor moderne
✓ Plăcile de rețea de pe laptopuri și PC-uri noi
✓ Routerele moderne

5. Concluzie

✓ **Cablurile Direct (Straight-Through)** sunt utilizate pentru conectarea **dispozitivelor diferite**.
✓ **Cablurile Încrucișate (Crossover)** sunt utilizate pentru conectarea **dispozitivelor similare**.
✓ **Auto-MDIX elimină necesitatea cablurilor crossover** în rețelele moderne.
✓ **Cat5e, Cat6 și Cat7** sunt cele mai utilizate cabluri Ethernet, fiecare având avantaje diferite.

Cablare UTP și STP – Diferențe și Utilizări

Cablurile folosite în rețele Ethernet sunt de obicei **cabluri torsadate (Twisted Pair)**, împărțite în două mari categorii:

- **UTP (Unshielded Twisted Pair)** – fără protecție împotriva interferențelor.
- **STP (Shielded Twisted Pair)** – cu protecție suplimentară împotriva interferențelor electromagnetice.

1. Cablul UTP (Unshielded Twisted Pair)

🔹 **Ce este?**

- Un cablu torsadat **fără ecranare** suplimentară împotriva interferențelor.
- Cele mai utilizate în rețelele Ethernet, fiind **mai ieftine și mai flexibile**.

🔹 **Caracteristici:**
✅ Cost redus și ușor de instalat.
✅ Utilizat în majoritatea rețelelor LAN.
✅ Poate fi afectat de interferențe electromagnetice (EMI) dacă este aproape de echipamente electrice puternice.

🔹 **Utilizare:**
✓ Rețele casnice și de birou (Ethernet, Cat5e, Cat6, Cat6a).
✓ Telefonie și cabluri de date.
✓ Camere de supraveghere (dacă nu sunt lângă surse de interferențe).

2. Cablul STP (Shielded Twisted Pair)

🔹 **Ce este?**

- Similar cu UTP, dar cu **protecție împotriva interferențelor**.
- Fiecare pereche de fire poate avea un strat de **ecranare metalică** (folie sau împletitură de cupru).

🔹 **Tipuri de ecranare STP:**

Tip	Descriere
FTP (Foiled Twisted Pair)	Are un singur strat de folie de aluminiu care învelește toate perechile de fire.
STP (Shielded Twisted Pair)	Fiecare pereche este protejată individual cu folie.
S/FTP (Shielded Foiled Twisted Pair)	Fiecare pereche are folie și întregul cablu are un strat suplimentar de protecție (ecranare dublă).

🔹 **Caracteristici:**
✅ Protecție împotriva interferențelor EMI și RF.
✅ Utilizat în medii industriale sau în zone cu echipamente electrice puternice.
✅ Mai scump și mai greu de instalat decât UTP.

🔹 **Utilizare:**
✓ Rețele din fabrici, spitale, aeroporturi (unde sunt multe interferențe).
✓ Cablare în apropierea cablurilor electrice de înaltă tensiune.
✓ Data centers și conexiuni de mare viteză.

3. Compararea UTP vs. STP

Caracteristică	UTP	STP
Protecție împotriva interferențelor	❌ Nu	✅ Da
Flexibilitate	✅ Ușor de instalat	❌ Mai rigid
Cost	✅ Mai ieftin	❌ Mai scump
Performanță în medii cu EMI ridicat	❌ Afectat de interferențe	✅ Funcționează bine
Distanță maximă fără pierderi	~100m	~100m (dar cu mai puține erori)

4. Concluzie

✓ **Cablul UTP** este alegerea ideală pentru **majoritatea rețelelor LAN** datorită costului redus și instalării ușoare.
✓ **Cablul STP** este recomandat **în medii industriale**, lângă echipamente electrice puternice, pentru a preveni interferențele.